

DataTube Architecture Review

目标存储模型

系统现在按 5 类路径工作，每一类只承担单一职责。

类别	设置项	主要内容
行情 SQLite 数据库路径	sqlite_db_path	Crypto / Stock 宽表快照
本地订单数据库路径	order_list_db_path	本地下单记录、持仓回补
策略监控数据库路径	strategy_monitoring_db_path	策略监控原始行、参数编辑、策略详情
实时市场 SQLite 路径	market_realtime_db_path	markets_state / market_deltas
策略统计 SQLite 目录	strategy_metrics_db_dir	每条策略一个 .db ，按 Translation + row_id 命名

当前数据流

1. services/realtime_collector.py
写入 sqlite_db_path ，提供 Crypto / Finance 行情快照。
2. services/ws_market_sync_service.py + main.py
订阅 Polymarket WebSocket，写入 market_realtime_db_path 。
3. services/polymarket_service.py
从 strategy_monitoring_db_path 读取策略监控行，从 market_realtime_db_path 读取实时盘口快照，并计算策略展示字段。
4. services/strategy_event_service.py
从 market_realtime_db_path 读取 market_deltas 生成策略事件流。
5. services/strategy_chart_service.py
从 market_realtime_db_path 读取 Polymarket 价格序列，从 sqlite_db_path 读取 Crypto / Finance overlay。
6. services/strategy_stats_store.py
在 strategy_metrics_db_dir 下为每条策略维护独立统计库。

本次重构完成的事情

- 统一实时市场 SQLite 读写路径为 `market_realtime_db_path`。
- 停止把 `sqlite_db_path` 当成实时市场库的 fallback。
- 保留对旧字段 `strategy_option_sqlite_db_path` 的读取兼容，但不再作为正式设计输出。
- 统一 WebSocket 监控流名为 `StrategyMonitoring`，替代 `MyFavorites`。
- 取消实时市场 SQLite 的 `favourites` 分库写入，所有 `markets_state` / `market_deltas` 统一落同一个实时市场库。
- `ws_market_sync_service.py` 改为在启动时解析真实的策略监控表名，避免 `monitoring` 与实际表名不一致。

代码职责映射

1. 行情 SQLite

- `services/realtime_collector.py`
- `services/sqlite_store.py`
- `services/strategy_chart_service.py` 中的 `_market_overlay_db_path()`

2. 本地订单数据库

- `services/strategy_profit_engine.py`
- `services/polymarket_service.py` 中的持仓回补逻辑

3. 策略监控数据库

- `services/polymarket_service.py`
- `services/strategy_profit_engine.py`
- `main.py` 中的策略监控候选加载

4. 实时市场 SQLite

- `main.py` 中的 `WebSocketMonitor`
- `services/ws_market_sync_service.py`
- `services/polymarket_service.py`
- `services/strategy_event_service.py`
- `services/strategy_chart_service.py`

5. 策略统计 SQLite 目录

- `services/strategy_stats_store.py`
- `services/polymarket_service.py` 中的统计同步入口

当前仍保留的兼容层

- `config.json` 已迁到 `strategy_monitoring`，`main.py` 仍兼容读取旧 `favorites` 块。
- `main.py` 里仍保留 `load_my_favorites()` 兼容包装，内部已统一转到 `load_strategy_monitoring_candidates()`。
- `config_loader.py` 仍能读取旧 `strategy_option_sqlite_db_path`，但保存时会清理掉。

这些兼容层是为了避免现有运行入口和旧配置立即失效，不再代表推荐设计。

当前架构评价

优点

- 5 类数据库职责已经可以在代码层明确区分。
- 策略图表、事件、详情都能围绕同一个实时市场 SQLite 工作。
- 策略统计目录设计清晰，是当前最稳定的一块。

已修复问题

- 实时市场库与行情库混用。
- `MyFavorites` 语义漂移。
- 策略监控表名被运行时错误覆盖。
- Web UI 与 `main.py` 直接运行时路径逻辑不一致。

剩余风险

- 若外部脚本仍写入旧 `favorites` 配置块，运行时虽然兼容，但建议一并迁移。
- MongoDB 集合命名已跟随新流名变化，旧历史集合不会自动迁移。
- `web_settings.json` 里的旧字段会在下一次保存设置时被清理，但现有文件可能暂时还能看到遗留键。

建议的后续方向

1. 把 `config.json` 中的 `favorites` 段整体迁移成 `strategy_monitoring`。
2. 给实时市场 SQLite 补一份轻量 `schema` 文档，明确 `markets_state` / `market_deltas` 字段约定。
3. 若后续不再需要 MongoDB，可继续收敛为单一 SQLite 架构。